

devops2-endterm

Single-node Kubernetes platform · mka1ybek · 2026-05-18

1. Topology

- **Nodes:** 1 (single-node, control-plane untainted to schedule workloads)
- **Hardware:** Ubuntu 25.10 VM on Apple Silicon Mac (UTM), 4 vCPU · 7.2 GiB RAM · 32 GiB disk · ARM64
- **Host IP:** 172.20.10.4
- **CRI:** containerd 2.2.1
- **Kubernetes:** v1.29.15 (kubespray-installed)
- **Pod CIDR:** 10.233.64.0/18 · **Service CIDR:** 10.233.0.0/18

```
Mac —ssh→ VM 172.20.10.4 → node1 (Ready, control-plane)
                                |
                                ├── etcd (127.0.0.1:2379, TLS)
                                ├── containerd → all pods
                                └── kubelet, kube-proxy (iptables)
```

2. Deployment process

Stack: Terraform → Ansible → Kubespray → ArgoCD (app-of-apps)

```
terraform apply -var=run_bootstrap=true
```

- generates ansible/inventory.ini + kubespray inventory
- triggers ansible-playbook site.yml
 - prepare-vm.yml (apt, swap, sysctl, kernel modules)
 - kubespray cluster.yml (etcd + control-plane + CNI)
 - fetch kubeconfig
 - untaint master + PSA namespace labels
 - install ArgoCD + apply root-app
 - seal DB credentials (one-time)

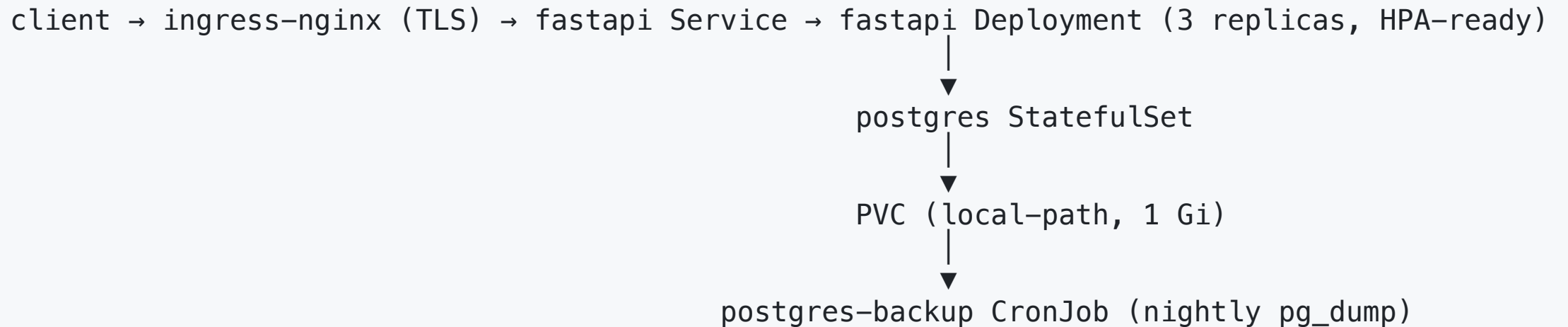
One command from cold metal to a fully-bootstrapped GitOps cluster.

3. Access & configuration management

- **Source of truth:** git → ArgoCD (`prune: true`, `selfHeal: true`)
- **App-of-apps:** `argocd/root-app.yaml` walks `argocd/apps/` → 12 child Applications, ordered by `sync-wave` annotations (0..10)
- **Admin access:** kubespray-issued client cert in `/etc/kubernetes/admin.conf`, fetched to Mac at `chmod 600` — no passwords
- **Non-admin access:** `dev-readonly` Role + RoleBinding (in `charts/business/templates/rbac.yaml`); per-user kubeconfigs via CSR + cert signing (documented in `docs/access.md`)
- **Secrets:** Bitnami **sealed-secrets** — only public key on dev laptop, plaintext never enters git

4. Business system architecture

Use case: small FastAPI HTTP service backed by PostgreSQL — the kind of stack a real team would actually run on this cluster.



Helm chart at `charts/business` : 26 resources — app, db, configmap, sealed secret, ingress, network policies, RBAC, PSA-restricted namespace, quotas, limits, PDB, backup, alerts.

5. Security of the workload

Pod-level

- `runAsNonRoot: true`, `runAsUser: 1000` (fastapi), 999 (postgres)
- `readOnlyRootFilesystem: true` · `/tmp` mounted as `emptyDir`
- `allowPrivilegeEscalation: false` · `capabilities.drop: [ALL]`
- `seccompProfile.type: RuntimeDefault` (pod + container)

Namespace-level

- `ResourceQuota` : 2 CPU / 2 GiB requests, 4 CPU / 4 GiB limits, 20 pods, 5 PVCs
- `LimitRange` : default requests/limits for containers without `resources:`
- `PodDisruptionBudget` : `minAvailable: 1` on fastapi

Cluster-level

6. Networking

Layer	Component
CNI	Calico (only CNI that enforces <code>NetworkPolicy</code> egress)
kube-proxy	iptables
DNS	CoreDNS + nodelocaldns (per-node 169.254.25.10 cache)
Ingress	ingress-nginx (NodePort 30443)
TLS	cert-manager + self-signed cluster CA <code>cluster-ca-issuer</code> — every Ingress carries <code>cert-manager.io/cluster-issuer</code> and <code>ssl-redirect: true</code>
LB	none (single-node) — Mac uses SSH port-forward

NetworkPolicy posture (business ns): default-deny ingress + egress plus 6 explicit

7. Storage

Concern	Implementation
StorageClass	local-path-provisioner (Rancher) — default class
Postgres data	<code>StatefulSet</code> → 1 Gi PVC via <code>volumeClaimTemplate</code>
Prometheus	5 Gi PVC
Alertmanager	1 Gi PVC
Grafana	2 Gi PVC
Loki	local-path PVC
Backup	nightly <code>pg_dump</code> CronJob → dedicated 2 Gi <code>postgres-backup</code> PVC, retains 7 dumps
Replication	n/a — single node by assignment

8. Rollout strategy

Deployments (fastapi):

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0    # never drop below desired replicas
    maxSurge: 1
readinessProbe: { httpGet: /ready, periodSeconds: 5 }
```

- ArgoCD picks up image-tag bumps from git → rolls out with zero downtime
- `scripts/demo-zero-downtime.sh` proves 100% HTTP-200 throughout a rolling update

Testing in CI (`.github/workflows/security-scan.yml`):

- Trivy image scan (CRITICAL gate)
- Trivy IaC scan (audit)

9. Custom extensions

Type	What	Where
Custom Helm chart	<code>business</code> — 26 templated resources (app, db, RBAC, netpol, PSA ns, backup, alerts, quotas)	<code>charts/business/</code>
	5 ClusterPolicies: disallow-latest, require-	

10. Monitoring

Stack: kube-prometheus-stack + loki-stack

Component	Purpose
Prometheus	metrics scrape + 7d retention on 5 Gi PVC
Alertmanager	alert routing, 1 Gi PVC
Grafana	dashboards, TLS ingress at <code>grafana.local</code> , admin pw from <code>existingSecret: grafana-admin</code>
kube-state-metrics	cluster object state
node-exporter	host CPU/mem/disk/net
Loki + Promtail	container logs, queryable from Grafana datasource
	certificates for ingress (added by <code>cert-manager</code>)

Verify any claim live

```
export KUBECONFIG=$(pwd)/kubeconfig
```

```
kubectl get nodes -o wide
```

```
kubectl -n argocd get app
```

```
# 12 Applications
```

```
kubectl get clusterpolicy
```

```
# 5 Kyverno policies
```

```
kubectl get clusterissuer
```

```
# cert-manager root CA
```

```
kubectl get networkpolicy -A
```

```
# 7 in business ns
```

```
kubectl get ns -L pod-security.kubernetes.io/enforce
```

```
kubectl -n business get cronjob postgres-backup
```

```
kubectl get prometheusrule -A
```

```
bash scripts/demo-netpol.sh
```

```
# NetworkPolicy proof
```

```
bash scripts/demo-zero-downtime.sh
```

```
# rolling update proof
```

Thank you

`github.com/mkalybek/devops2-endterm`